

WTW Software Project Proposal

CSE 416 Software Engineering | Fall 2026

What's the Word?

Figure out the move without the group-chat debate.

Team	Project
Alan Razin Josh Mike	WTW — What's the Word?

WTW helps friends move from “What should we do?” to one agreed plan through private preference collection, group ranking, and a short final vote.

1 Project and Product Thesis

Short pitch

WTW is a social decision-making application for friends who want to decide where to eat or what to do without an extended group-chat debate. It turns a vague conversation into a short workflow: choose a group, evaluate a limited set of nearby options, compare the group's preferences, and select one plan.

Product thesis

Maps and review apps are designed to help one person discover places. WTW addresses a different job: helping several people agree. Its core value is not a larger catalog. It is a fair and efficient path from many possibilities to one decision.

Nearby options → 20–30 candidates → Top three → One plan

Project outcome

The semester project will deliver a deployed, mobile-first web application in which authenticated or invited users can create a group Pick, privately rate a shared candidate pool, receive an explainable ranking, cast a final vote, and save the result.

2 Problem and Existing Alternatives

Problem

Group plans often stall even when everyone wants to meet. Members may have real preferences but hesitate to propose them, the first suggestion can anchor the discussion, and each new idea creates another round of messages. The group is trying to solve two tasks at once: discover acceptable options and negotiate agreement.

WTW separates those tasks. It supplies a bounded set of relevant choices, collects opinions independently, and reveals only the combined result. This gives quieter members equal input and prevents the group from replacing one endless conversation with an endless feed.

Current alternatives

Alternative	What it does well	Gap WTW addresses
Group chat	Everyone already has access and can discuss details.	Suggestions are sequential, preferences are visible, and no mechanism forces convergence.
Google Maps or Yelp	Strong place search, reviews, hours, photos, and directions.	Designed mainly for individual discovery rather than private group agreement.
General polls	Collect votes quickly when choices are already known.	Someone must research and enter the options, and the poll lacks live place context.
One person decides	Fast when the group accepts a designated chooser.	The choice may not reflect the group and often defaults to familiar places.

WTW's differentiator is the complete decision loop: generate relevant choices, collect private preferences, rank agreement, and finish with one selected plan.

3 Target Users and Roles

Primary user

The initial audience is college students and young adults making casual plans with two to eight friends. Stony Brook students are the first practical testing community because the team can recruit participants, watch groups use the workflow, and revise the product within the semester.

User roles

Role	Responsibilities and capabilities
Host	Starts a Pick, selects the group and category, accepts default constraints or refines them, invites participants, and closes the response period.
Participant	Joins an authorized Pick, rates candidates privately, monitors completion status, votes among the finalists, and views the winner.
Individual user	Maintains a profile and preferences, manages friends and groups, browses a bounded Discover feed, and reviews prior decisions.
System	Retrieves candidate data, enforces permissions and Pick states, aggregates responses, ranks finalists, synchronizes progress, and stores the outcome.

4 Requirements in One View

User	Need	Requirement	Usability condition
Friends making a plan	Reach agreement without a long chat.	Create a Pick, rate candidates, view three finalists, vote, and receive one winner.	A routine Pick starts in under 30 seconds and targets a decision in about five minutes.
Invited participant	Join without confusion or lost work.	Open the correct Pick, submit private responses, and resume later.	Saved responses survive refresh and temporary disconnection.
Returning group	Avoid rebuilding membership each time.	Save a group and use it for multiple independent Picks.	Membership and history persist with server-side authorization.
Individual user	Discover and save ideas outside a live Pick.	Browse a bounded feed using location and basic preferences.	Discover supports the group workflow without becoming an infinite feed.

5 Core User Experience

First-time onboarding

1. Create an account and basic profile.
2. Answer a short preference survey covering food, activities, nightlife, budget, and distance.
3. Grant location access or enter a search area manually.
4. Add friends through the app or share an invitation link.

Onboarding provides a useful starting signal; it is not intended to build a perfect taste profile. Later swipes and completed Picks can refine preferences if the team reaches the personalization phase.

Primary navigation

Area	Purpose
Discover	A bounded individual feed for browsing nearby food and activities, adjusting simple preferences, and saving ideas.
Friends	Friend relationships, reusable groups, active Picks, invitations, and completed group decisions.
Profile	Account settings, preference controls, saved places, and personal activity history.

Group Pick

1. The host opens a saved group or selects individual friends.
2. The host chooses Food or Activities. Current location, open now, and a reasonable radius are defaults; detailed filters are optional.
3. WTW retrieves a common pool of approximately 20 to 30 candidates and invites the participants.
4. Each participant independently marks every candidate Yes, Maybe, or No. Candidate order may vary, but individual answers remain hidden.
5. WTW saves progress and shows who has finished without revealing opinions.
6. When the response period closes, WTW calculates the top three group matches and explains the ranking.
7. Participants vote among the finalists. The original group score resolves a tie.
8. WTW displays one winner with details and directions, then saves the result to group history.

Independent browsing means hidden answers and potentially randomized order, not unrelated candidate pools. Participants must evaluate the same options for the group score to be meaningful.

6 Version 1 Scope

Committed scope

- Account creation, sign-in, profiles, and a short preference survey.
- Friend relationships, reusable groups, invitation links, and basic blocking.
- A limited Discover experience for food and selected activities.
- Pick creation, participant tracking, saved progress, cancellation, and server-controlled lifecycle states.
- External place search with normalization, filtering, caching, mock data, and failure handling.
- Private Yes, Maybe, and No responses across a shared candidate pool.
- An explainable top-three ranking, real-time completion status, final voting, and one saved winner.
- Responsive mobile layouts, accessibility basics, automated tests, deployment, and monitoring.

Explicitly outside version 1 but we plan on trying to get it done

- Native iOS and Android applications, reservations, payments, ride sharing, and ticket purchasing.
- A public social feed, a full reviews platform, or a complete replacement for group messaging.
- Weather-aware ranking, advanced midpoint routing, calendar integration, and hundreds of categories.
- Machine-learning personalization as a dependency of the core decision workflow.

Stretch features

A lightweight group chat, Never Been Mode, Adventure Mode, preference profiles, midpoint search, local events, and weather-aware recommendations remain optional. The team will select at most one major stretch feature after the core decision loop is stable.

7 Functional Requirements

ID	Actor	Requirement	Need
FR-01	User	Create an account, sign in, sign out, and update a profile.	Persistent identity
FR-02	User	Complete and edit a short preference survey.	Relevant starting options
FR-03	User	Send, accept, remove, and block friend relationships.	Usable social graph
FR-04	User	Create, name, and manage a reusable group.	Repeated decisions
FR-05	Host	Create a Pick for selected friends or a saved group.	Begin a decision
FR-06	Host	Choose Food or Activities and optionally refine location, radius, price, and open status.	Relevant constraints
FR-07	System	Retrieve and normalize a bounded shared candidate pool.	Avoid cold start and endless browsing
FR-08	Participant	Join an authorized Pick through the app or an invitation link.	Low-friction participation
FR-09	Participant	Record and revise one Yes, Maybe, or No response per candidate before closure.	Private preference capture
FR-10	System	Persist responses and restore progress after refresh or reconnection.	Reliable participation
FR-11	System	Hide individual responses until the group ranking is produced.	Independent judgment
FR-12	Host	Close responses when everyone finishes or according to a documented timeout rule.	Prevent stalled Picks
FR-13	System	Calculate deterministic scores and return up to three finalists.	Group agreement
FR-14	System	Explain each finalist without exposing a participant's individual answer.	Trust and privacy
FR-15	Participant	Cast one final vote for an eligible finalist.	Final group choice
FR-16	System	Resolve ties, declare one winner, and store the outcome.	Complete the plan
FR-17	User	Browse a bounded individual Discover feed and save ideas.	Immediate individual value

ID	Actor	Requirement	Need
FR-18	Group member	View completed Picks for groups they belong to.	Continuity

8 Nonfunctional Requirements

ID	Quality	Measurable requirement
NFR-01	Performance	Cached candidate retrieval completes within two seconds for at least 95 percent of expected-load test requests.
NFR-02	Performance	A preference submission is acknowledged within one second for at least 95 percent of expected-load test requests.
NFR-03	Reliability	A saved response remains available after refresh, sign-out and sign-in, or temporary disconnection.
NFR-04	Authorization	Every protected action validates the user and the relevant Pick or group membership on the server.
NFR-05	Privacy	Participants cannot retrieve another participant's individual responses during swiping.
NFR-06	Integrity	Repeated requests do not create duplicate participants, preferences, or final votes.
NFR-07	Accessibility	Primary workflows are keyboard operable and provide labeled controls, visible focus, and adequate contrast.
NFR-08	Responsiveness	Core screens remain usable at viewport widths of 360 pixels and above.
NFR-09	Maintainability	Core services expose typed contracts and automated tests; external place data is isolated behind a provider interface.
NFR-10	Observability	Production errors, Pick completions, and external API failures are logged without recording private response contents.

9 User Stories and Acceptance Criteria

ID	User story	Acceptance criterion
US-01	As a student, I want to start a Food Pick with my roommates so that we can choose quickly.	From a saved group, the host launches a default Pick in no more than three primary actions.
US-02	As an invited friend, I want my opinions hidden while I rate options so that I am not influenced by the group.	No participant-facing response reveals another person's Yes, Maybe, or No before ranking.

ID	User story	Acceptance criterion
US-03	As a participant, I want my progress saved so that I do not repeat my choices after closing the app.	Returning to the Pick restores every saved response.
US-04	As a group member, I want a short list of strong matches so that the decision does not become another feed.	After closure, the system returns exactly three finalists when at least three valid candidates exist.
US-05	As a participant, I want a final vote so that the group ends with one plan.	Each authorized participant votes once; the system declares and stores one winner.
US-06	As a returning user, I want to reuse my group so that I do not rebuild it for every plan.	The same saved group can start multiple independent Picks and view its completed history.

10 Ranking and Decision Logic

Version 1 uses a transparent scoring model rather than presenting the result as an artificial-intelligence prediction. Each response maps to a utility value: Yes = 1.0, Maybe = 0.5, and No = 0.0.

$$\text{WTW Score} = 100 \times (0.70 \times \text{preference} + 0.20 \times \text{consensus} + 0.10 \times \text{coverage}) - 15 \times \text{no-vote proportion}$$

The weights are a starting hypothesis. The team will compare alternatives with unit tests and user feedback. The displayed result will be labeled a WTW Match Score, not a probability.

Test case	Decision question
Three Yes, one No	Should broad enthusiasm outweigh one strong objection?
Four Maybe responses	Should complete moderate agreement beat a more divisive option?
Missing response	How much should incomplete coverage reduce the score?
Equal group scores	Which deterministic tie-breaker should run first?
Tied final vote	Can the original WTW Score resolve the result consistently?

11 Architecture and Data

WTW will use a conventional web architecture so the team can focus on product behavior, correctness, and integration. A responsive web application or progressive web application supports invitation links and avoids app-store approval during a short semester.

Client	Application services	Data and external systems
React and Next.js Mobile-first interface Client interaction state	Typed server API Authorization Pick state machine Ranking service Provider adapter	PostgreSQL Authentication Real-time updates Place-data provider Monitoring

The place provider will sit behind a WTW-owned interface. Production may use Google Places or another approved service, while development and tests use a seeded mock provider. This keeps provider fields out of the rest of the application and makes quotas, caching, and failures testable.

Core data

Entity	Purpose
User and Friendship	Identity, profile, preferences, accepted relationships, and blocks.
Group and GroupMember	Reusable group membership and roles.
Pick and PickParticipant	Category, location, constraints, host, lifecycle state, invitation, and completion status.
Candidate and PickCandidate	Normalized place record and the historical snapshot used for one Pick.
Preference and RankingResult	Private response, computed score, ranking position, and explanation.
FinalVote and PickOutcome	Final ballot, tie resolution, selected winner, and completion time.
Notification	Invitation and Pick-status notification state.

Pick lifecycle

The server controls the sequence Draft → Swiping → Ranking → Final Vote → Completed. A Pick may also become Canceled. The server rejects actions that do not belong to the current state, such as a late swipe after ranking or a vote for a non-finalist.

12 Why This Requires a Semester

A small prototype could swipe through hard-coded cards and calculate a local score. The proposed product must make that interaction reliable across real users and real data. Its difficulty lies in the connections between systems rather than the number of screens.

Engineering concern	Semester-level work
Multi-user state	Synchronize progress and final voting across clients while keeping individual preferences private.
Lifecycle correctness	Enforce valid Pick transitions and protect state from concurrent or duplicate actions.
External data	Normalize provider fields, manage quotas, cache candidates, and recover from provider failure.
Ranking	Define, test, explain, and revise group agreement and tie behavior.
Security	Protect invitations, memberships, host-only actions, histories, and private responses.
Fault handling	Support refresh, reconnection, late joins, missing participants, and simultaneous submissions.

Engineering concern	Semester-level work
Validation	Measure time to decision, completion, invitation acceptance, abandonment, and repeat use.
Delivery	Maintain environments, CI, tests, deployment, monitoring, and presentation-ready evidence.

13 Scope Decisions and Trade-offs

Decision	Alternative	Reason and trade-off
Build the group Pick loop first.	Develop Discover, chat, reviews, events, and personalization at equal depth.	A narrower core ends in a real decision. Secondary features remain available if the loop is stable.
Begin with a responsive web application.	Build separate native mobile applications.	Links and deployment are easier; native push and deeper device integration are postponed.
Use an explainable scoring model.	Claim machine-learning recommendations before behavioral data exists.	The initial result is testable and defensible; personalization can use real data later.
Use external place data through an adapter.	Maintain a local place directory.	External data avoids a content cold start; the adapter reduces vendor coupling and enables mocks.
Treat group chat as a stretch feature.	Make chat part of the first end-to-end release.	WTW must solve the decision, not spend the semester rebuilding a messaging product.

14 Team Responsibilities

The team will divide primary ownership by subsystem so that each member has a clear contribution. Each member will lead the design, implementation, testing, and documentation of their assigned area while working with teammates during integration and review.

Member	Primary ownership	Major deliverable
Alan	Pick lifecycle, state transitions, private response collection, preference aggregation, group ranking, tie-breaking, ranking explanations, and algorithm testing.	Produce three ranked finalists and one final outcome from the participants' responses.
Razin	Authentication, profiles, friend requests, reusable groups, group membership, invitations, authorization, blocking, and security testing	Allow users to securely create groups, invite participants, and access only authorized Picks.
Josh	Place-provider integration, geolocation, data normalization, category mapping, filtering, caching, API quota management, mock data, and provider-failure handling	Convert a Pick's location, category, and filters into a reliable pool of relevant nearby options.

Member	Primary ownership	Major deliverable
Mike	Mobile-first interface, onboarding, Discover, Pick creation, swiping, participant progress, real-time updates, final voting, winner screen, and accessibility	Allow users to complete the full experience from creating or joining a Pick to viewing the winner.

Integration work will be shared across subsystem boundaries. Alan and Razin will connect groups with Pick creation; Alan and Josh will connect Pick constraints with place discovery; Alan and Mike will connect Pick state with the real-time interface; Razin and Mike will develop the invitation and joining flow; and Josh and Mike will connect place data with candidate cards.

All four members will participate in requirements decisions, architecture and code reviews, integration testing, user testing, deployment, presentation preparation, and AI-use documentation. Every major feature will have a primary owner, a reviewer, automated tests, and at least two team members who understand how it works.

15 Phased Delivery

Phase	Working result
1 Proposal and thin slice	Finalize requirements and schema; establish the repository and CI; demonstrate a seeded Pick that accepts swipes, runs a ranking function, and displays finalists.
2 Identity and groups	Deliver authentication, profiles, friends, reusable groups, migrations, seed data, and a deployed development environment.
3 Pick lifecycle	Deliver creation, invitations, authorization, participants, saved progress, closure, cancellation, and enforced state transitions.
4 Place discovery	Deliver geolocation, provider adapter, normalization, filters, caching, mock data, usage monitoring, and failure handling.
5 Course MVP	Deliver multi-user private responses, real-time progress, tested ranking, top three, final vote, and one winner.
6 Product completion	Deliver Discover, history, notifications, mobile polish, accessibility, security review, and end-to-end tests.
7 Validation and launch	Conduct usability and performance testing, deploy production, add monitoring and analytics, repair failures, and prepare the final demonstration.
Final user window	Freeze risky features, monitor reliability and cost, recruit users, and collect evidence required by the course.

Course MVP

The MVP is complete when four users can enter the same Pick, rate one shared candidate pool privately, resume after refresh, see completion progress, receive a top three, cast final votes, and view one stored winner. Advanced personalization, push notifications, chat, and numerous categories are not required for the MVP.

16 Validation

Area	Evidence
Functional	A group completes account setup, group creation, Pick launch, invitation, private responses, ranking, final voting, and history.
Algorithm	Tests cover agreement, divisive candidates, incomplete responses, group sizes, ranking ties, and final-vote ties.
Security	Tests reject unauthorized reads, duplicate submissions, invalid candidates, invalid state transitions, expired invitations, and blocked users.
Reliability	Responses survive refresh and reconnection; provider failure and simultaneous actions do not corrupt a Pick.
Product	Measure time to create a Pick, time to winner, invitation acceptance, Pick completion, abandonment stage, and repeat group use.
Usability	Observe Stony Brook groups completing the workflow and compare it with their normal planning process.

The problem description is currently based on the team's product observation. Before the final proposal is submitted, the team should conduct short interviews or a survey and add only results that were actually collected.

17 Milestone 1 Demonstration

Milestone 1 must include a running artifact, not only a future plan. The team will demonstrate a thin vertical slice using seeded restaurant data:

1. Open the working WTW prototype.
2. Start a Food Pick for a mock group.
3. Submit Yes, Maybe, and No responses on several candidate cards.
4. Run the initial ranking function and display a top three.
5. Show at least one automated test for the ranking logic.
6. Identify which components are real, which use mock data, and what will be integrated next.

The presentation will also explain two decisions: prioritizing the group Pick loop over chat and broader discovery, and choosing a responsive web application before native mobile development.

18 AI Use and Accountability

AI may assist with research, design alternatives, scaffolding, implementation, testing, debugging, and documentation. It does not replace the team's responsibility for product decisions, security, verification, or oral explanation.

- AI-assisted commits will state what AI contributed, what the author changed, and how the result was tested.
- Comments will document significant AI assistance in non-obvious logic such as ranking, authorization, concurrency, caching, migrations, and complex tests.
- Before merge, the author must be able to explain the code's inputs, outputs, data effects, failure cases, security implications, and tests.
- Substantial AI-assisted changes will receive review from another team member.

Example commit note

AI-assisted: used an AI tool to draft the initial ranking implementation and suggest edge-case tests. I revised the weights, incomplete-response behavior, and tie-breaking rules, then verified the change with unit tests and peer review.